Security assessment

Target: OWASP crAPI v1.1.6, local stand (127.0.0.1)
Date: 2026-11-01
Method: OWASP WSTG, ASVS, API Security Top 10 (2023)

1. Executive summary

Fifteen vulnerabilities were reproduced by hand on the local crAPI stand and each is covered by an automated regression test. Four are rated critical: three different ways to bypass authentication entirely, and an unthrottled one-time-password endpoint that allows account takeover by brute force. One high-severity SQL injection returns other users' data, and a server-side request forgery lets the application be used to reach services on its internal network.

The findings chain: the community feed hands out e-mail addresses and vehicle identifiers, which are exactly the inputs the authentication and object-level findings need.

Severity	Findings
critical	4
high	2
medium	9
low	0

Last run of the blocking gate: **PASS** (2026-11-01T02:51:56+00:00), blocking 0 of 0 findings (no findings).

Scope and rules

Only containers of the Compose project `qa-crapi` on the loopback interface were tested. Server-side request forgery was directed exclusively at a canary container inside the stand's network. Brute-force demonstrations were limited to 20 attempts against accounts created by the test suite.

Risk heat map

severity \ category	API1	API2	API3	API5	API7	API8
critical		4				
high					1	1
medium	3		2	3		1
low						

2. Findings overview

ID	Title	OWASP API Top 10 (2023)	CWE	CVSS 3.1
FINDING-001	All three services accept a JWT with `alg: none`	API2 Broken Authentication	CWE-347	9.1 critical
FINDING-002	Identity accepts a token signed with the published demo secret	API2	CWE-321	9.1 critical
FINDING-015	Identity accepts a genuine token whose payload was edited	API2	CWE-347	9.1 critical
FINDING-012	No limit on OTP attempts in `/auth/v2/check-otp`	API2	CWE-307	9.1 critical
FINDING-003	SQL injection in `apply_coupon`	API8 Security Misconfiguration	CWE-89	8.8 high
FINDING-005	SSRF: `contact_mechanic` fetches any URL supplied by the caller	API7 SSRF	CWE-918	7.7 high
FINDING-006	Any user can read any vehicle's live location	API1 BOLA	CWE-639	6.5 medium
FINDING-007	Any user can read any order by id	API1 BOLA	CWE-639	6.5 medium
FINDING-008	Any user can read any mechanic report by id	API1 BOLA	CWE-639	6.5 medium
FINDING-009	A customer can delete any user's video through the admin route	API5 BFLA	CWE-285	6.5 medium
FINDING-010	The management user list is open to any authenticated user	API5 BFLA	CWE-285	6.5 medium
FINDING-004	NoSQL injection in coupon validation	API8	CWE-943	5.4 medium
FINDING-013	Service reports can be created without authentication	API5	CWE-306	5.3 medium
FINDING-011	Community feed exposes other users' e-mail and vehicle id	API3 Broken Object Property Level Authorization	CWE-213	4.3 medium
FINDING-014	A customer can set the fulfilment status of their order	API3 (mass assignment)	CWE-915	4.3 medium

3. Findings in detail

Fifteen findings reproduced by hand on 2026-11-01 against `http://127.0.0.1:8888`, each with a regression test in `security_tests/` that asserts the **secure** behaviour and is a strict expected failure while the stand is vulnerable. Scores are CVSS v3.1 base scores computed by `tools/cvss.py` (checked against published NVD vectors in `tests/test_cvss.py`), rated for crAPI **as if it were a real product**.

crAPI is intentionally vulnerable; these are its training vulnerabilities, not defects to report upstream.

ID	Title	OWASP API Top 10 (2023)	CWE	CVSS 3.1	Test
FINDING-001	All three services accept a JWT with `alg: none`	API2 Broken Authentication	CWE-347	9.1 critical	`test_unsigned_token_is_rejected`
FINDING-002	Identity accepts a token signed with the published demo secret	API2	CWE-321	9.1 critical	`test_token_signed_with_the_published_demo_secret_is_rejected`
FINDING-015	Identity accepts a genuine token whose payload was edited	API2	CWE-347	9.1 critical	`test_identity_rejects_a_tampered_token`
FINDING-012	No limit on OTP attempts in `/auth/v2/check-otp`	API2	CWE-307	9.1 critical	`test_otp_check_is_rate_limited`
FINDING-003	SQL injection in `apply_coupon`	API8 Security Misconfiguration	CWE-89	8.8 high	`test_coupon_code_is_not_concatenated_into_sql`
FINDING-005	SSRF: `contact_mechanic` fetches any URL supplied by the caller	API7 SSRF	CWE-918	7.7 high	`test_contact_mechanic_does_not_fetch_a_caller_supplied_url`
FINDING-006	Any user can read any vehicle's live location	API1 BOLA	CWE-639	6.5 medium	`test_vehicle_location_of_another_user_is_refused`
FINDING-007	Any user can read any order by id	API1 BOLA	CWE-639	6.5 medium	`test_order_of_another_user_is_refused`
FINDING-008	Any user can read any mechanic report by id	API1 BOLA	CWE-639	6.5 medium	`test_mechanic_report_of_another_user_is_refused`
FINDING-009	A customer can delete any user's video through the admin route	API5 BFLA	CWE-285	6.5 medium	`test_admin_video_route_is_refused_to_a_customer`
FINDING-010	The management user list is open to any authenticated user	API5 BFLA	CWE-285	6.5 medium	`test_management_user_list_is_refused_to_a_customer`
FINDING-004	NoSQL injection in coupon validation	API8	CWE-943	5.4 medium	`test_coupon_validation_rejects_mongo_operators`
FINDING-013	Service reports can be created without authentication	API5	CWE-306	5.3 medium	`test_service_report_creation_requires_authentication`
FINDING-011	Community feed exposes other users' e-mail and vehicle id	API3 Broken Object Property Level Authorization	CWE-213	4.3 medium	`test_community_posts_do_not_expose_personal_data`
FINDING-014	A customer can set the fulfilment status of their order	API3 (mass assignment)	CWE-915	4.3 medium	`test_order_status_is_not_client_writable`

Chains worth noting: **FINDING-011 → FINDING-012** (the feed hands out the e-mail addresses that the unthrottled OTP endpoint needs) and **FINDING-006/008 → FINDING-003** (identifiers and phone numbers read from one endpoint feed the next).

What crAPI gets **right** is tested too, as positive controls: anonymous requests are refused, the community and workshop services verify the token's algorithm and signature, another user's order cannot be modified, and `/auth/v3/check-otp` throttles after ten attempts. Those ten tests must pass, so a change that breaks them is caught as a regression.

FINDING-001

All three services accept a JWT with `alg: none` · API2:2023 · CWE-347 · CVSS 3.1 `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N` = **9.1 critical**

Endpoint / precondition. `identity`, `community` and `workshop`; the attacker needs only a victim's e-mail address, which the community feed publishes (FINDING-011).

Reproduction (localhost only):

```
header  = base64url({"alg": "none", "typ": "JWT"})
payload = base64url({"sub": "qa-bob@example.com", "role": "user", "iat": ..., "exp": ...})
token   = f"{header}.{payload}."          # empty signature
GET /identity/api/v2/user/dashboard        Authorization: Bearer <token>   -> 200
GET /community/api/v2/community/posts/recent                               -> 200
GET /workshop/api/shop/orders/all                                          -> 200
```

Evidence. The identity dashboard returns Bob's name, e-mail, phone number and credit balance; the workshop returns Bob's orders. Genuine tokens are RS256 (`{"alg":"RS256"}` in the login response).

Impact. Complete authentication bypass as any user, without a password, a stolen token or any interaction. Everything the other findings require a session for becomes reachable.

Fix. Pin the accepted algorithm to RS256 and reject tokens whose header says otherwise; never call a decoder with signature verification disabled. The crAPI source shows the pattern Semgrep also flagged (`unverified-jwt-decode` in `services/workshop`, `jwt-go-parse-unverified` in `services/community`).

Regression test. `security_tests/test_authentication.py::test_unsigned_token_is_rejected` (per service).

FINDING-002

Identity accepts a token signed with the published demo secret · API2:2023 · CWE-321 · CVSS 3.1 `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N` = **9.1 critical**

Reproduction. Sign the same claims with HS256 and the key `crapi` — the value shipped as `JWT_SECRET` in crAPI's own compose file — and call `GET /identity/api/v2/user/dashboard`: **200** with the victim's profile. The same token is refused by community and workshop (**401**), which is the positive control.

Impact. Account takeover of any user by anyone who has read the project's public configuration.

Fix. One key source, asymmetric verification only, no hard-coded or shipped secrets; rotate on deploy.

Regression test. `test_token_signed_with_the_published_demo_secret_is_rejected`, plus `test_community_and_workshop_reject_the_demo_secret` as the control.

FINDING-015

Identity accepts a genuine token whose payload was edited · API2:2023 · CWE-347 · CVSS 3.1 `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N` = **9.1 critical**

Same root cause as FINDING-002 — the identity service does not verify signatures at all — recorded separately because it is a different attack and a different test. Taking a valid session token, changing `role` to `admin` and keeping the original signature still returns **200**. Community and workshop refuse the same token.

Found by accident: this test was written as a positive control, assuming every service would refuse a broken signature. It failed, which is how the finding was discovered.

Regression test. `test_identity_rejects_a_tampered_token`; the control is `test_community_and_workshop_reject_a_tampered_token`.

FINDING-012

No limit on OTP attempts in `/auth/v2/check-otp` · API2:2023 · CWE-307 · CVSS 3.1 `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N` = **9.1 critical**

Reproduction (20 attempts against an account this suite created, per the rules of engagement): `POST /identity/api/auth/forget-password` then 20 × `POST /identity/api/auth/v2/check-otp` with wrong codes — all 20 answered **500**, none throttled. The same 20 attempts against `/auth/v3/check-otp` are blocked with **503** after the tenth.

Impact. The OTP is four digits, so the whole key space is 10 000 attempts; at the measured rate of ~33 requests/s that is roughly five minutes to take over an account whose e-mail is known — and the community feed publishes e-mail addresses (FINDING-011).

Fix. Retire v2; apply v3's throttling (and an attempt counter bound to the reset request) everywhere.

Regression test. `test_otp_check_is_rate_limited`; control `test_otp_v3_is_rate_limited`.

FINDING-003

SQL injection in `apply_coupon` · API8:2023 · CWE-89 · CVSS 3.1 `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H` = **8.8 high**

Reproduction. As any logged-in customer:

```
 POST /workshop/api/shop/apply_coupon
{"coupon_code": "'; SELECT number FROM user_login WHERE email='qa-bob@example.com", "amount": 75}

400 {"message":"9000000002 Coupon code is already claimed by you!! ..."}
```

`9000000002` is **another user's** phone number, read out of the `user_login` table and returned in the error message. A single quote alone (`"coupon_code": "'"`) returns a Django 500 error page, which is how the injection point was spotted.

Evidence from the source (`services/workshop/crapi/shop/views.py`, `ApplyCouponView`): the value is concatenated into `cursor.execute(...)`. Semgrep's `p/security-audit` pack did **not** flag this line; the finding came from the manual probe, which is the point of running both.

Impact. Arbitrary read of the application database (user table included) and, because the statement separator is accepted, arbitrary statements — hence integrity and availability are rated High as well.

Fix. Parameterised queries or the ORM; never interpolate request data into SQL; turn off detailed error pages in production.

Regression test. `test_coupon_code_is_not_concatenated_into_sql` (asserts that another user's phone number never appears in the response).

FINDING-005

SSRF: `contact_mechanic` fetches any URL supplied by the caller · API7:2023 · CWE-918 · CVSS 3.1 `AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N` = **7.7 high**

Reproduction. `POST /workshop/api/merchant/contact_mechanic` with `"mechanic_api": "http://canary:8080/ssrf-proof"`. The response embeds the canary's body (`{"canary":"QA-SSRF-CANARY-7f3c", ...}`) and the canary records the inbound request from the workshop container (`172.22.0.9`). **Only the stand's canary is ever used as a destination** (rules of engagement, guardrail 3).

Impact. The server fetches attacker-chosen URLs and returns the body, so anything the workshop container can reach — other services on the internal network, cloud metadata endpoints in a real deployment — is readable from outside. Scope is Changed

because the request crosses into the internal network.

Fix. Do not take an endpoint URL from the request; resolve the mechanic's address server-side from the mechanic code, and allow-list the hosts the service may call.

Regression test. `test_contact_mechanic_does_not_fetch_a_caller_supplied_url` (asserts the canary was never contacted).

FINDING-006

Any user can read any vehicle's live location · API1:2023 · CWE-639 · CVSS 3.1 `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N` = **6.5 medium**

`GET /identity/api/v2/vehicle/{uuid}/location` with Alice's token and Bob's vehicle id returns **200** with latitude, longitude, owner's full name and e-mail. Anonymous requests are correctly refused (**401**), so the check that exists is "is there a token", not "is this your car".

Impact. Physical tracking of any owner whose vehicle id is known; vehicle ids are published by the community feed (FINDING-011), so the identifier is not a secret.

Fix. Authorise on the object: the vehicle must belong to the caller.

Regression test. `test_vehicle_location_of_another_user_is_refused`; controls `test_own_vehicle_location_is_allowed` and `test_vehicle_location_requires_authentication`.

FINDING-007

Any user can read any order by id · API1:2023 · CWE-639 · CVSS 3.1 `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N` = **6.5 medium**

`GET /workshop/api/shop/orders/{id}` returns another customer's order: product, quantity, status, the buyer's e-mail and phone number, and the payment transaction id. Order ids are small integers, so the whole order history of the shop can be enumerated.

Notable asymmetry: `PUT` on the same order **is** authorised (**403**), which shows the ownership check exists and was simply not applied to the read path. That asymmetry is the positive control `test_changing_another_users_order_is_refused`.

Fix. Apply the same ownership check to reads; use unguessable identifiers.

Regression test. `test_order_of_another_user_is_refused`.

FINDING-008

Any user can read any mechanic report by id · API1:2023 · CWE-639 · CVSS 3.1 `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N` = **6.5 medium**

`GET /workshop/api/mechanic/mechanic_report?report_id=1` returns a report belonging to another customer: owner's e-mail and phone number, the vehicle's VIN and the free-text problem description (which in the seeded data contains the owner's own phone number and e-mail). Report ids are sequential.

Fix. Authorise by owner or assigned mechanic.

Regression test. `test_mechanic_report_of_another_user_is_refused`.

FINDING-009

A customer can delete any user's video through the admin route · API5:2023 · CWE-285 · CVSS 3.1 `AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N` = **6.5 medium**

`DELETE /identity/api/v2/admin/videos/{id}` with Bob's ordinary customer token deletes Alice's video: **200 "User video deleted successfully."** The route checks that a token exists, not that it carries an administrative role.

Impact. Destruction of other users' data by any registered customer. Rated I:H because the deletion is permanent for the affected object; no confidentiality impact.

Fix. Enforce the role on every `/admin/` route, server-side, in one place.

Regression test. `test_admin_video_route_is_refused_to_a_customer` (creates and cleans up its own video).

FINDING-010

The management user list is open to any authenticated user · API5:2023 · CWE-285 · CVSS 3.1 `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N` = **6.5 medium**

`GET /workshop/api/management/users/all` returns every user of the shop with e-mail, phone number and credit balance, to any logged-in customer.

Impact. Bulk harvesting of the user base: the input for the OTP brute force (FINDING-012) and for enumerating objects in the BOLA findings.

Fix. Role check plus pagination limits on administrative listings.

Regression test. `test_management_user_list_is_refused_to_a_customer`.

FINDING-004

NoSQL injection in coupon validation · API8:2023 · CWE-943 · CVSS 3.1 `AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N` = **5.4 medium**

`POST /community/api/v2/coupon/validate-coupon` with `{"coupon_code": {"$ne": "zzz"}}` (or `{"$gt": ""}`) returns a real coupon — `{"coupon_code":"TRAC075","amount":"75"}` — while an unknown string returns an error. A JSON object is passed into the MongoDB query where a string is expected.

Impact. Any customer can obtain valid discount coupons without knowing a code, and can confirm which coupons exist.

Fix. Validate the type of every field before it reaches the query; reject objects where a string is expected.

Regression test. `test_coupon_validation_rejects_mongo_operators` (both operators); control `test_unknown_coupon_is_rejected`.

FINDING-013

Service reports can be created without authentication · API5:2023 · CWE-306 · CVSS 3.1 `AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N` = **5.3 medium**

`GET /workshop/api/mechanic/receive_report` with no token creates a service request for an arbitrary VIN and returns a link to the new report. The OpenAPI specification also declares this operation without a security requirement, so the specification and the implementation agree — on being open.

Impact. Anyone who can reach the API can fill the mechanics' queue and create reports against other people's vehicles.

Fix. Require authentication, or restrict the route to the internal network if it is a service-to-service callback — and say so in the specification.

Regression test. `test_service_report_creation_requires_authentication`.

FINDING-011

Community feed exposes other users' e-mail and vehicle id · API3:2023 · CWE-213 · CVSS 3.1 `AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N` = **4.3 medium**

`GET /community/api/v2/community/posts/recent` returns, for every post, `author.email` and `author.vehicleid` next to the nickname. The web client shows only the nickname, so the data is invisible in the browser and present in the response.

Impact. The directory an attacker needs for the rest of the chain: e-mail addresses for the OTP brute force (FINDING-012) and vehicle ids for the location BOLA (FINDING-006).

Fix. Serialise only what the client renders; build responses from an explicit field list rather than dumping the model.

Regression test. `test_community_posts_do_not_expose_personal_data`.

FINDING-014

A customer can set the fulfilment status of their order · API3:2023 · CWE-915 · CVSS 3.1
`AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N` = **4.3 medium**

`PUT /workshop/api/shop/orders/{own id}` with `{"status": "returned"}` changes the stored status, which the shop treats as a return (the credit flow follows from it). The field is accepted straight from the request body.

Scope note. Only the caller's own order: the same request against another user's order is correctly refused with **403** (positive control). The finding is mass assignment, not a broken ownership check.

Fix. Bind only the fields a customer may change; move status transitions behind the return workflow.

Regression test. `test_order_status_is_not_client_writable` (creates its own order, so it does not depend on what other tests left behind).

Observations (not rated)

ID	Observation	Why it is recorded
OBS-1	The identity service answers **404** instead of 401 to an unauthenticated request for `/user/dashboard`	The request is refused, so it is not a vulnerability, but the status misleads clients and monitoring. The regression tests accept 401, 403 or 404 as "refused" and say why.
OBS-2	The LLM chatbot challenges are not covered	`crapi-chatbot` needs an external OpenAI key and is removed from the stand (ADR-002)
OBS-3	Shell injection through video conversion parameters was not attempted	The stand sets `ENABLE_SHELL_INJECTION=false`; with it enabled the host-side risk is out of proportion to the demonstration

4. Static analysis

Four tools, two targets: **this repository**, which the team owns, and **crAPI at v1.1.6**, a third-party application that is vulnerable on purpose. What blocks a build and what is only reported is decided in ADR-003.

Tool	Version (pinned by digest)	Target	Result	Gate
Gitleaks	v8.30.1	git history of 5 portfolio repositories (26 commits)	0 leaks	blocking
Semgrep	1.183.0	this repository (32 files, 405 rules)	0 findings	blocking
Trivy	0.76.1	this repository (dependencies, config, secrets)	0 findings	blocking
Semgrep	1.183.0	crAPI source, 374 files	54 findings	report
Trivy / Syft / Grype	0.76.1 / v1.55.0 / v0.121.0	6 crAPI images	see below	report

Secrets (Gitleaks)

Full history of `devsecops-quality-gates`, `shop-sut`, `shop-e2e-playwright`, `shop-api-pytest` and `shop-performance-k6`: **0 leaks**. Test credentials in these repositories live in `.env.example` files and are documented as test-only, which is why they are not matched by the default rules.

The gate is demonstrated to fail on a planted fake token: gate-demo.md.

This repository (Semgrep, Trivy)

Semgrep with `p/python`, `p/security-audit`, `p/secrets`, `p/dockerfile` and `p/docker-compose`: **0 findings** over 32 files. The default ignore list is replaced by `.semgrepignore` so test code is scanned as well.

Trivy over the repository reported nothing. Worth stating plainly: this repository contains **no file type Trivy's misconfiguration scanner supports** (no Dockerfile, Kubernetes, Terraform or CloudFormation), and `uv.lock` had no vulnerable package at the time of the scan. An empty result is "nothing to find here", not "everything was checked".

crAPI source (Semgrep, report only)

54 findings over 374 files. Grouped:

Count	Rule	Service	Triage
36	`request-host-used`	web	**False positive** for our purpose: Nginx/Lua config reading the request host; no fix applies to a target we do not own
4	`disabled-cert-validation`	workshop	**True positive**, CWE-295: `verify=False` on outbound HTTPS, including the call that performs FINDING-005
3	`missing-user`	community, identity, web	**True positive**, CWE-250: container images run as root
2	`missing-ssl-minversion`	community	**True positive**, CWE-326
2	`python-logger-credential-disclosure`	chatbot	Out of scope: the chatbot is not deployed on this stand (ADR-002)
1	`unverified-jwt-decode`	workshop	**True positive**, CWE-347 — the static counterpart of FINDING-001
1	`jwt-go-parse-unverified`	community	**True positive**, CWE-347 — same class
1	`insecure-hostname-verifier`	identity	**True positive**, CWE-297
1	`spring-csrf-disabled`	identity	True positive in general; low value for a token-authenticated API
1	`no-csrf-exempt`	workshop	Same
1	`math-random-used`	gateway-service	**True positive**, CWE-338, in a stub service
1	`avoid_app_run_with_bad_host`	chatbot	Out of scope

What SAST missed. The SQL injection of FINDING-003 — `cursor.execute` with a concatenated request value in `services/workshop/crapi/shop/views.py` — was **not** reported: zero rules matching `sql` fired across the 374 files. In the same file Semgrep flagged only line 141 (`verify=False`). The finding came from a manual probe that sent a single quote and got a 500. This is the clearest argument in the project for keeping manual testing next to the scanners.

crAPI images (Trivy, Syft, Grype — report only)

Six service images, scanned from the registry by digest, `linux/amd64`. SBOMs in CycloneDX format are produced by Syft and are the input Grype scans, so the two tools see the same inventory.

Image	Tool	Critical	High	Medium	Low	Other	Critical with a fix
api.mypremiumdealership.com	Trivy	9	107	190	132	8	8
api.mypremiumdealership.com	Grype	21	177	153	23	67	19
crapi-community	Trivy	3	79	108	63	7	3
crapi-community	Grype	36	152	118	21	7	26
crapi-identity	Trivy	14	60	532	167	0	14
crapi-identity	Grype	14	67	794	258	3	12
crapi-web	Trivy	2	57	85	64	6	2
crapi-web	Grype	33	165	134	29	7	23
crapi-workshop	Trivy	10	190	182	85	6	10
crapi-workshop	Grype	34	276	234	51	7	24
mailhog	Trivy	3	55	71	56	3	3
mailhog	Grype	20	97	77	19	3	20

Overlap by advisory and package (GHSA ids mapped to their CVE alias so one advisory is not counted twice):

Image	Found by both	Trivy only	Grype only
api.mypremiumdealership.com	433	13	8
crapi-community	259	1	75
crapi-identity	651	122	485
crapi-web	213	1	155
crapi-workshop	434	39	168
mailhog	187	1	29

Reading the comparison. The tools agree on the bulk of the findings and disagree at the edges, in two ways that matter when a gate depends on them:

1. **Severity is not the same number.** Grype reports several times more Critical findings than Trivy on the same image (crapi-web: 33 vs 2) because the tools prefer different sources for the rating — vendor data where Trivy has it, the NVD score where Grype does not. A gate that blocks on "Critical" therefore blocks on a different set depending on the tool, which is an argument for pinning one tool per gate and recording why.
2. **Coverage differs by ecosystem.** Grype finds hundreds of extras in the Java image (crapi-identity), Trivy finds 122 the SBOM scan does not — scanning the image directly sees files an SBOM may omit, and vice versa.

None of this blocks a build here: these are a third-party target's images (ADR-003). The numbers are kept so that the next scan can be compared with this one, and `tools/sca_summary.py` regenerates the tables.

Reproduce

```
make scan-secrets          # Gitleaks over the portfolio repositories
make scan-sast             # Semgrep: own code (gate) and crAPI source (report)
make scan-sca              # Trivy repo + images, Syft SBOMs, Grype, and the summary tables
make gate                  # the blocking gate over the own-code reports
```

5. Dynamic analysis and coverage

Three automated layers ran against the same stand as the manual testing: Semgrep over crAPI's source, ZAP's Automation Framework against the running API, and Nuclei. This page compares their output with the fifteen findings in findings.md, which were reproduced by hand.

The short version

Layer	Tool	Findings reported	Of the 15 manual findings, how many	Max severity
SAST	Semgrep 1.183.0, 374 source files	54	**2 partially** (`unverified-jwt-decode`, `jwt-go-parse-unverified` point at FINDING-001)	ERROR
DAST	ZAP 2.17.0, API policy, 105 endpoints	9 alert types	**0**	Medium
DAST	Nuclei 3.11.1, 3 546 templates	24 (all one template, informational)	**0**	Info
Manual + regression tests	—	15	15	Critical

The scanners did not find a single one of the authorization findings, the SQL injection, the SSRF or the missing rate limit. They did find nine classes of issue that the manual tests never looked for.

ZAP

Configuration: `stand/zap-plan.yaml` — context restricted to `http://crapi-web`, the pinned OpenAPI specification imported (44 URLs added) and the spider limited to depth 5 (96 URLs found); the report counts 105 endpoints in the end. Active scan with the `API` policy and a 12-minute budget (it finished in 1 m 54 s), authenticated by injecting a bearer token through a replacer rule. Reports: HTML, JSON and SARIF in `reports/zap/`.

Alert	Risk	Instances
Content Security Policy (CSP) Header Not Set	Medium	5
Cross-Domain Misconfiguration	Medium	5
Missing Anti-clickjacking Header	Medium	1
Server leaks version information via the `Server` header	Low	5
Application Error Disclosure	Low	5
X-Content-Type-Options Header Missing	Low	5
Timestamp Disclosure	Low	2
Modern Web Application / Authentication Request Identified	Info	5

A caveat the report itself records: **85 % of the responses ZAP saw were 4xx** (7 % 2xx, 4 % 5xx). Generic payloads against a strongly typed API are mostly rejected before any logic runs, which is part of why the active scan found nothing — and a reason not to read "0 findings" as "nothing to find".

Worth noticing: ZAP did report **Application Error Disclosure (Low)** — the same class of leaked error page that gave away the SQL injection during manual testing — but on five *other* endpoints (`receive_report`, `shop/orders`, `login-with-token`, `check-otp` v2 and v3), not on `apply_coupon`. So the scanner found the symptom class and rated it Low, and never produced it at the one endpoint where it meant an injection. Getting there needs a request body the API accepts and a coupon code it recognises; the active scanner's generic payloads did not reach that state.

Nuclei

Templates tagged `misconfig`, `exposure`, `tech` and `default-login` against three entry points, rate limited to 20 requests/s: **0 matches** in 621 requests. The second pass (`-id http-missing-security-headers`) reports 24 matches across the three entry points — ten header types, all informational — and exists as a self-check: an empty result from the first pass now means "nothing matched", not "the scanner never reached the target".

That distinction was learned the hard way. The first Nuclei run reported 0 findings and had in fact scanned nothing: nuclei resolves hostnames with its own resolvers and could not see Docker's internal DNS (`Skipped crapi-web:80 ... no address found for host`). The fix is `-system-resolvers`. **A scanner that reports nothing is indistinguishable from a scanner that ran nothing, unless you make it prove otherwise.**

Semgrep

54 findings over crAPI's source, triaged in findings-static.md. Two of them point at the root cause of FINDING-001 (`unverified-jwt-decode` in the workshop, `jwt-go-parse-unverified` in the community service) — the only overlap between the automated layers and the manual findings.

Semgrep **missed the SQL injection**: zero rules matching `sql` fired across 374 files, although `services/workshop/crapi/shop/views.py` concatenates a request value into `cursor.execute`. In that same file it reported only `verify=False` on an outbound request.

Why the gap exists

Finding class	Why a scanner misses it
BOLA / BFLA (FINDING-006…010)	The scanner has one identity and no notion of "whose object is this". A 200 for `/vehicle/{other id}/location` looks like a successful request, not a violation. Detecting it needs two accounts and an expectation about ownership — which is what the regression tests encode.
Mass assignment (FINDING-014)	Requires knowing which fields a client may legitimately set; the API accepts the request and answers 200.
Excessive data exposure (FINDING-011)	Requires knowing that `email` is not meant to be in a public feed. The response is valid JSON with a 200.
Missing rate limit (FINDING-012)	A scanner avoids sending hundreds of identical requests by design; and the absence of a response is the signal.
SSRF (FINDING-005)	Needs an out-of-band listener the scanner controls and can verify. ZAP can do this with OAST, which was deliberately not enabled: it would send traffic to an external service, outside the rules of engagement. The project uses a local canary instead.
SQL injection (FINDING-003)	Here it is a scanner-tuning failure rather than a conceptual one. The endpoint needs a valid session, a JSON body and a coupon code the API accepts, and the evidence is a value echoed into a 400 message — not a timing difference or a stack trace.

What this means for the pipeline

1. **Scanners are a floor, not a ceiling.** They cover the classes nobody should have to re-test by hand (headers, known CVEs, outdated components) and they run on every change. Everything that depends on business meaning — ownership, roles, which field belongs to whom — needs tests written by someone who knows the domain.
2. **Both layers are gated differently** (ADR-003): new High or Critical from the scanners blocks, and the fifteen regression tests must keep their exact strict-xfail status, so a change in either direction is caught.
3. **Every scanner needs a self-check.** Nuclei got an explicit one; ZAP's equivalent is that its report must contain the URLs from the OpenAPI import, and Semgrep's is the rule count in its SARIF output.

Reproduce

```
make stand-up && make seed
make scan-dast      # ZAP (~4 min incl. 2 min active scan) and Nuclei (~1 min)
make gate-target    # the DAST gate against stand/target-baseline.yml
```